

MEMSYST MASTER DEVELOPMENT DIRECTIVE

Technical Architecture, Engineering Principles & Implementation Standards

This document is the foundational instruction set for any AI Agent, Software Engineer, Technical Architect, Development Team, or Future Contributor working on the MemSyst Platform.

This directive must

resume task was a mistake

Understood. I'll continue and complete the prompt.

MEMSYST MASTER DEVELOPMENT DIRECTIVE

Technical Architecture, Engineering Principles & Implementation Standards

This document is the foundational instruction set for any AI Agent, Software Engineer, Technical Architect, Development Team, or Future Contributor working on the MemSyst Platform.

This directive must be reviewed and followed before any development work begins. All implementation decisions must align with the business objectives, architecture principles, governance requirements, and long-term vision defined herein.

1. PROJECT OVERVIEW

What is MemSyst?

MemSyst is a multi-tenant Organizational Operating System designed to provide digital infrastructure for:

Professional Associations

Membership Organizations

Trade Associations

Cooperatives

Unions

Federations

Training Institutions

Industry Bodies

Membership-Based Organizations

MemSyst is not generic business software.

MemSyst is purpose-built to help organizations manage:

Membership
Governance
Finance
Training
Communication
Events
Certification
Identity
Marketplace Activities
Support Programs
Advertising
Analytics
Operations

through a unified digital infrastructure platform.

2. BUSINESS-FIRST DEVELOPMENT PRINCIPLE

The MemSyst platform is a business platform, not a technology experiment.

All technical decisions must support:

Business Agility
Operational Efficiency
Tenant Self-Service
Security
Performance
Affordability
Scalability
Maintainability
Long-Term Sustainability

Technology must serve business objectives.

Technology choices that increase complexity without delivering measurable business value should be avoided.

3. CURRENT DEVELOPMENT STRATEGY

Phase 1 Strategy

The objective is to:

Launch Quickly
Validate the Business Model
Acquire Tenants
Generate Revenue
Build Core Infrastructure
Establish Market Presence

The platform should not be over-engineered during initial development.

4. OFFICIAL TECHNOLOGY APPROACH

Firebase-First Architecture

The initial implementation shall use:

Authentication

Firebase Authentication

Database

Cloud Firestore

File Storage

Firebase Storage

Notifications

Firebase Cloud Messaging (FCM)

Backend Services

Firebase Cloud Functions

Hosting

Firebase Hosting

Why Firebase?

The objective is:

Rapid Development
Low Infrastructure Cost
Automatic Scaling
Built-In Security
High Availability

Reduced DevOps Overhead

Fast Time-To-Market

Firebase is the initial platform infrastructure, not the permanent architecture.

5. FUTURE-PROOF ARCHITECTURE REQUIREMENT

Critical Rule

The platform must never be tightly coupled to Firebase.

All business logic must be developed using abstraction layers.

Developers must implement:

Service Layer Architecture

Repository Pattern

Domain-Based Modules

API Abstraction Layer

Business logic must not directly depend on Firebase APIs.

Reason

Future platform growth may require:

PostgreSQL

Supabase

Dedicated APIs

Dedicated Backend Services

Data Warehousing

Microservices

Enterprise Integrations

The platform must be capable of evolving without requiring a complete rebuild.

6. RECOMMENDED APPLICATION STACK

Frontend

Framework

Next.js

Requirements:

App Router
TypeScript
Server Components where appropriate
SEO Optimization
Performance Optimization

UI Framework

Tailwind CSS

Component System

ShadCN UI

Requirements:

Reusable Components
Accessible Components
Consistent Design Patterns
Maintainable UI Architecture

7. MULTI-TENANCY REQUIREMENTS

MemSyst is a multi-tenant platform.

Tenant isolation is mandatory.

Required Tenant Structure

Platform
└ Tenant
 └ National
 └ Region
 └ Branch
 └ Member

Every Record Must Include

tenantId

Where applicable also include:

regionId
branchId
memberId

Examples:

```
{  
  "tenantId": "ghaba",  
  "regionId": "ashanti",  
  "branchId": "kumasi"  
}
```

No module may be developed without tenant isolation.

8. CONTENT MANAGEMENT PHILOSOPHY

Tenant Self-Service First

A core objective of MemSyst is:

Organizations should manage their own content without requiring software developers.

Therefore

Business content must not be hardcoded.

Tenant administrators should be able to manage:

Homepage Content
About Pages
Executive Profiles
Membership Categories
Training Programs
Events
Support Programs
Directories
Certificates
ID Templates
Fee Structures
Advertisements
Marketplace Categories
Public Information Pages

through administrative interfaces.

Not code.

9. PLATFORM CONFIGURABILITY PRINCIPLE

Wherever possible:

Configuration > Hardcoding

Examples:

Approval Workflows

Membership Fees

Renewal Fees

Withdrawal Policies

Revenue Distribution

Advertisement Pricing

Permissions

Notifications

Templates

must be configurable through administration dashboards.

10. SECURITY PRINCIPLES

Core Principle

Trust Nothing

Validate Everything

Audit Everything

Security Layers

Authentication

Firebase Authentication

Authorization

Role-Based Access Control (RBAC)

Tenant Isolation

Mandatory.

Audit Logging

Mandatory.

Financial Audit Trails

Immutable.

Data Validation

Server-side validation required.

Rate Limiting

Protect:

Login

Registration

Payments

Messaging

Public Forms

11. FINANCIAL SYSTEM PRINCIPLES

Official Payment Gateway

Paystack

Only.

Wallet Funding

Users fund wallets using Paystack.

Example:

Real Cash GH¢100

↓

Verified by Paystack

↓

Wallet Credit GH¢100 Units

Wallet Rules

No Manual Funding

No Peer-To-Peer Transfers

No Member-To-Member Transfers

No Cash Handling Inside Platform

Financial Records

Must use:

Double Entry Ledger Principles

Immutable Transactions

Full Audit Trails

12. PERFORMANCE REQUIREMENTS

The platform must be designed for speed from the beginning.

Requirements:

Pagination

Lazy Loading

Caching

Optimized Queries

Image Optimization

CDN Delivery

Efficient Firestore Usage

Avoid:

Full Collection Reads

Unbounded Queries

Duplicate Data Fetching

13. DEVELOPMENT QUALITY STANDARDS

All code must be:

Modular
Reusable
Documented
Typed
Testable
Maintainable

Avoid

Hardcoded Values
Business Logic in UI Components
Duplicated Logic
Monolithic Files
Direct Firebase Coupling

14. STAGE-BASED DEVELOPMENT APPROACH

Development follows the official MemSyst implementation roadmap.

The current immediate priority is:

Stage 1

MemSyst Business Operations Platform (M-BOP)

Stage 1 includes:

Admin Dashboard
Form Management
Lead Management
CRM Pipeline
Notifications
Tenant Onboarding
Basic User Management
Audit Logs

All future stages depend on Stage 1.

Do not build future-stage functionality prematurely.

15. AI AGENT IMPLEMENTATION RULE

Before implementing any feature:

The AI Agent must:

First

Review:

This Master Development Directive

Second

Review:

Current Stage Specification

Third

Implement only the scope of the current stage.

The AI Agent Must Not

Invent Business Rules

Change Approved Workflows

Create Unapproved Features

Modify Revenue Logic

Alter Governance Rules

without explicit instruction.

16. LONG-TERM PLATFORM VISION

MemSyst is being built as a long-term digital infrastructure platform capable of serving:

Thousands of Organizations

Millions of Members

Multiple Industries

Multiple Countries

The architecture must therefore prioritize:

Scalability

Maintainability

Security

Configurability

Affordability

Operational Simplicity

while remaining practical for a startup-stage business.

FINAL DEVELOPMENT DIRECTIVE

MemSyst shall initially adopt a Firebase-First Architecture using Firebase Authentication, Firestore, Firebase Storage, Firebase Cloud Messaging, Firebase Cloud Functions, and Next.js. The platform must prioritize affordability, maintainability, security, performance, tenant self-service content management, configurability, and rapid business deployment. All business logic must be implemented through abstraction layers to avoid vendor lock-in and allow future migration to PostgreSQL, Supabase, dedicated backend services, or microservice architectures without requiring major platform redesign. Every development decision must support the business objectives of MemSyst as a scalable Organizational Operating System and Digital Infrastructure Platform.

MemSyst Architecture Directive

Technology Independence, Cost Optimization, Scalability & Maintainability

This directive supplements the existing **Memsyst Platform Architecture** and must be treated as a core architectural requirement for all current and future development.

Objective

Design and implement MemSyst as a platform that **uses Firebase as an infrastructure service rather than being tightly coupled to Firebase-specific implementations.**

The architecture must prioritize:

- Long-term maintainability
- Cost efficiency
- Security
- High performance
- Scalability
- Vendor flexibility
- Ease of future migration
- Multi-tenant SaaS operation
- Low operational overhead

The platform must remain capable of evolving beyond Firebase in the future without requiring major application rewrites.

Core Architectural Principle

Technology Independence Principle

Business functionality must never depend directly on Firebase SDKs, Firestore structures, Cloud Functions, or any vendor-specific implementation.

All business modules must communicate through defined service and repository layers.

The application must be architected so that Firebase can be replaced in the future with alternative technologies (PostgreSQL, Supabase, AWS, Azure, etc.) with minimal impact on business logic and user-facing functionality.

Required Architecture Layers

Implement a strict layered architecture.

Presentation Layer (UI)

↓

Application Layer
(Use Cases / Services)

↓

Domain Layer
(Business Rules & Workflows)

↓

Repository Layer
(Data Access Contracts)

↓

Infrastructure Layer
(Firebase, Storage, External Services)

Rules

- UI components must never call Firebase directly.
- UI components must never contain business logic.
- Business workflows must never be implemented inside pages or components.
- Business logic must never depend on Firebase SDKs.
- Repository interfaces must shield the application from infrastructure implementation details.

Repository Pattern Requirement

All database operations must pass through repository contracts.

Example:

MemberService

↓

MemberRepository Interface

↓

FirestoreMemberRepository

Never allow:

Page

↓

Firestore

or

Component

↓

Firebase SDK

directly.

Repository interfaces must support future implementation replacements such as:

- Firestore
- PostgreSQL
- Supabase
- REST APIs
- GraphQL APIs

without affecting business modules.

Business Logic Isolation

All business rules must reside within dedicated domain services.

Examples:

- Membership approval workflow
- Apprentice registration workflow
- Apprentice transfer workflow
- Fee calculations
- Membership renewals
- Executive approval workflows
- Training certification workflows

Business rules must not be embedded in:

- React Components
- Next.js Pages
- Firebase Functions
- Database Queries

Cloud Functions may execute workflows but must not contain the workflow definitions themselves.

Multi-Tenant Architecture Standard

Maintain a single shared platform architecture.

Do not create separate Firebase projects per tenant.

All tenant data must be isolated through:

tenantId

organizationId

and security enforcement.

Every tenant-aware entity must support:

tenantId

createdBy

updatedBy

createdAt

updatedAt

status

where applicable.

Cost Optimization Requirements

The platform must be designed to minimize Firebase consumption.

Query Optimization

All list views must support:

- Pagination
- Filtering
- Search
- Indexed queries

Avoid:

Load entire collections

or

Unbounded reads

at all costs.

Data Aggregation Strategy

Reports must not repeatedly scan large datasets.

Instead:

Operational Data

↓

Aggregation Process

↓

Summary Collections

↓

Reports

Reports should consume pre-calculated summaries whenever possible.

Caching Strategy

Implement caching where appropriate.

Examples:

- Organization settings
- Membership categories
- Branch configurations
- Public content
- Dashboard summary data

Prioritize reducing repeated Firestore reads.

Storage Optimization

Do not store large files in Firestore.

Store files using:

- Firebase Storage

Design storage abstraction so future migration to:

- Cloudflare R2
- AWS S3
- Azure Blob Storage

is possible without affecting business modules.

Authentication Strategy

Use Firebase Authentication as the primary identity provider.

Preferred authentication methods:

- Email & Password
- Email Verification
- Password Reset

Avoid making SMS-based authentication the primary login mechanism due to long-term cost implications.

All authentication logic must be abstracted behind authentication services.

Cloud Function Guidelines

Cloud Functions are infrastructure execution tools, not business logic containers.

Cloud Functions may:

- Trigger workflows
- Process background jobs
- Send notifications
- Generate documents
- Schedule tasks

Cloud Functions must not contain critical business rules.

Business rules must exist within reusable domain services.

Future Database Flexibility

The architecture must support future integration of relational databases.

Particularly for:

- Finance
- Accounting
- Payments
- Reporting
- Analytics

Prepare interfaces that allow future migration of selected modules to:

PostgreSQL

without impacting other platform modules.

Do not hardcode Firestore assumptions into business workflows.

Reporting & Analytics Strategy

Design reporting architecture with future expansion in mind.

Future path:

Operational Data

↓

Aggregated Data

↓

Analytics Layer

↓

BigQuery / Data Warehouse

Avoid designing reports that depend on large-scale real-time Firestore scans.

Security Requirements

All development must follow:

1. Least privilege access.
 2. Tenant-level data isolation.
 3. Server-side authorization validation.
 4. Role-based access control.
 5. Audit logging.
 6. Secure API boundaries.
 7. Firestore Security Rules enforcement.
 8. No direct client access to privileged operations.
-

Cloudflare Readiness

The platform architecture must be compatible with future adoption of:

- Cloudflare CDN
- Cloudflare Caching
- Cloudflare Security Services
- Edge Performance Optimizations

Avoid architectural decisions that would prevent future Cloudflare integration.

Development Standards

For every module:

Sequence

Database Design

↓

Security Model

↓

Repository Contracts

↓

CRUD Operations

↓

Business Rules

↓

Workflow Engine

↓

Dashboard Integration

↓

Reporting

↓

Notifications

↓

Audit Logging

↓

UI Refinement

This sequence is mandatory.

Deliverable Expectation

Before implementing any module, the AI agent must ensure:

- No direct Firebase dependency exists in UI components.
- Repository abstraction exists.
- Business logic is isolated.
- Multi-tenant support is enforced.
- Future database replacement is possible.
- Firebase usage is optimized.
- Reporting is scalable.
- Security architecture is enforced.
- Long-term maintainability is prioritized over short-term development speed.

The resulting MemSyst platform must be Firebase-first, but not Firebase-dependent. It should remain secure, affordable, scalable, maintainable, and capable of evolving into a large multi-tenant digital infrastructure platform without requiring architectural redesign.

MemSyst Financial Infrastructure & Revenue Model Directive

Purpose

This directive defines the financial architecture, revenue model, fee management framework, and sustainability requirements for the MemSyst Platform.

The objective is to ensure that MemSyst operates as a financially sustainable multi-tenant SaaS platform while providing organizations with flexible fee configuration and revenue generation capabilities.

This directive must be incorporated into the Financial Infrastructure Module (Phase 2 – Stage 5) and treated as a core platform requirement.

Financial Architecture Principles

The financial system must support:

- Platform sustainability
- Multi-tenant operations
- Configurable organizational fees
- Revenue sharing
- Platform commissions
- Transaction processing
- Wallet management
- Auditability
- Financial reporting
- Future payment gateway integrations

All financial operations must be fully auditable and traceable.

No financial transaction may be deleted.

Financial records must support complete historical tracking.

Revenue Model Structure

The MemSyst revenue model shall consist of four revenue layers:

Layer 1 – Platform Subscription Revenue

This is the primary recurring revenue source.

Organizations may be assigned subscription plans such as:

Starter

- Monthly Fee
- Quarterly Fee
- Annual Fee

Growth

- Monthly Fee
- Quarterly Fee

- Annual Fee

Enterprise

- Custom Pricing

The platform must support:

- Plan creation
- Plan assignment
- Plan upgrades
- Plan downgrades
- Plan renewals
- Trial periods
- Grace periods
- Subscription suspension

Subscription plans must be configurable by Super Admin.

Layer 2 – Platform Commission Revenue

The platform shall support configurable commissions on selected transactions.

Initial supported categories:

Accounts

- Account Registration
- Account Renewal
- Account Transfer
- Account Upgrade

Print Services

- QR Code ID Card Printing
- QR Code Certificate Printing

Content & Listings

- Marketplace Posts
- Training Posts
- Other Listing Categories
- Sponsored/Advertising Posts

Wallet Transactions

- Wallet Funding Processing
- Wallet Withdrawals

The default configuration shall support percentage-based commissions.

Example:

Account Registration	10%
Account Renewal	10%
Account Transfer	10%
Account Upgrade	10%

QR ID Card Printing	10%
QR Certificate Printing	10%

Marketplace Posts	10%
Training Posts	10%
Other Categories	10%

Advertising Posts	100%
-------------------	------

Wallet Funding	5%
Wallet Withdrawal	5%

All percentages must be configurable.

No values should be hardcoded.

Layer 3 – Communication Revenue Recovery

Communication services must support cost recovery and optional markup.

Supported channels:

- Email
- SMS
- Push Notifications
- In-App Notifications
- OTP Services

The platform must support:

Cost Price

Actual provider cost.

Charge Price

Amount charged to tenant.

Markup

Platform profit margin.

Example:

SMS Cost: 0.30

SMS Charge: 0.50

Markup: 0.20

The system must separately track:

- Communication costs
- Communication revenue
- Communication profit

Note:

Email, Push, and In-App notifications should primarily support cost recovery and operational management rather than act as primary profit centers.

Layer 4 – Advertising Revenue

Advertising must operate as a dedicated revenue stream.

Supported advertising products may include:

- Homepage Banner Ads
- Marketplace Ads
- Training Hub Ads
- Directory Featured Listings
- Sponsored Content
- Announcement Promotions

The platform must support:

- Fixed pricing
- Duration pricing

- Impression pricing (future)
- Click pricing (future)
- Sponsorship packages

Advertising revenue shall be tracked separately from other revenue categories.

Fee Configuration Engine

Implement a centralized Fee Configuration Engine.

All platform fees must be managed from a single administration interface.

Features required:

- Create Fee Category
- Create Fee Type
- Percentage-Based Fees
- Fixed Amount Fees
- Hybrid Fees
- Effective Dates
- Fee Activation/Deactivation
- Tenant Overrides
- Historical Fee Tracking

Examples:

Registration Fee

Renewal Fee

Transfer Fee

Upgrade Fee

Wallet Fee

Marketplace Fee

Training Fee

Advertising Fee

Notification Fee

No fee logic should be hardcoded within modules.

All modules must retrieve fee values from the Fee Configuration Engine.

Revenue Allocation Framework

The financial architecture must support revenue allocation rules.

Each transaction should allow allocation to:

Tenant

Branch

Regional Office

National Office

Platform Operator (MemSyst)

Partner Organization

Allocation rules must be configurable.

Future organizations may use different revenue-sharing structures.

Wallet & Transaction Processing

The Wallet Infrastructure must support:

Wallet Funding

- Online Payments
- Mobile Money
- Bank Transfers
- Future Integrations

Wallet Usage

- Fee Payments
- Purchases
- Registrations
- Renewals

Wallet Withdrawals

- Tenant Withdrawals
- Marketplace Payouts
- Vendor Payouts

The platform may apply configurable withdrawal commissions.

Financial Ledger Requirements

Every financial activity must create ledger entries.

Supported records:

- Debits
- Credits
- Adjustments
- Refunds
- Reversals
- Commission Deductions
- Revenue Allocations

The ledger shall serve as the financial source of truth.

No transaction should exist without corresponding ledger records.

Reporting Requirements

Provide financial reporting at:

Platform Level

- Total Revenue
- Subscription Revenue
- Commission Revenue
- Advertising Revenue
- Communication Revenue
- Outstanding Balances

Tenant Level

- Income
- Expenses
- Wallet Activity
- Membership Revenue
- Marketplace Revenue
- Training Revenue

Operational Reports

- Daily Revenue
- Monthly Revenue

- Annual Revenue
 - Revenue by Organization
 - Revenue by Category
 - Revenue by Branch
 - Revenue by Region
-

Audit & Compliance Requirements

Every financial operation must be auditable.

Track:

- User
- Tenant
- Date
- Time
- Previous Value
- New Value
- Transaction Source
- Approval History

Financial records must never be physically deleted.

Use status-based archival only.

Future Expansion Readiness

The architecture must be designed to support future additions including:

- Payment Gateways
- Mobile Money Providers
- Banking Integrations
- Escrow Services
- Installment Payments
- Subscription Billing Automation
- Tax Management
- Invoicing
- Advanced Accounting

- Financial Analytics

The design must remain provider-agnostic and support future migration to PostgreSQL or other financial-grade data stores without impacting business workflows.

Implementation Rule

Before building any financial feature, development must follow:

Financial Data Model

↓

Security & Permissions

↓

Ledger Architecture

↓

Fee Configuration Engine

↓

Wallet Infrastructure

↓

Transaction Processing

↓

Revenue Allocation

↓

Reporting

↓

Notifications

↓

Audit Logging

↓

UI Refinement

The resulting MemSyst Financial Infrastructure must provide a sustainable recurring revenue model for the platform while giving tenant organizations complete flexibility to manage their own fees, transactions, revenue streams, and financial operations.